UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 01 2025/2026

Subject	:	Database Programming (SECP3623)
Section	:	Section 02
Task	:	Project I
Due	:	28th November 2025

Name	Matric No.
Muhammad Syahmi Faris bin Rusli	A23CS0138
Muhammad Afiq Danish bin Mohd Hazni	A23CS0118
Pravinraj A/L Sivabathi	A23CS0171
Dheshieghan A/L Saravana Moorthy	A23CS0072

Presentation Video Link: Video Project 1 - Group 4

Table of Contents

1.0 Introduction

1.1 Chosen Domain

For this project, our group decided to work on a hostel room reservation system. We chose this domain because room booking is something most students deal with every semester, and we realised that a lot of the process is still manual, confusing, or inconsistent. By designing a proper database for it, the whole flow of registering students, checking room availability, and handling payments can be made much more organised. A solid backend is important because it ensures that the information stored is accurate, easy to retrieve, and protected from errors.

1.2 Objectives of the System

The main goal of the system is to support common hostel-related tasks using a clean and reliable database. We wanted to create tables that can handle student details, room information, reservations, and payments without causing conflicts or duplicated data. At the same time, the database should allow different SQL operations such as filtering, sorting, aggregating, and joining data when needed. Another objective is to demonstrate how indexing can improve query performance, especially when the dataset grows larger. Overall, the system serves as a complete backend example that ties together everything we learned in the course.

1.3 Team Members and Roles

Member Name	Roles
Muhammad Afiq Danish bin Mohd Hazni	Database Designer (DDL)
Dheshieghan A/L Saravana Moorthy	Data Manipulation & Query Developer (DML + DQL)
Muhammad Syahmi Faris bin Rusli	Query Specialist (DQL)
Pravinraj A/L Sivabathi	Indexing & Optimization Analyst

2.0 SQL Implementation Summary

This project's backend was developed entirely using SQL, focusing on writing and executing DDL, DML as well as DQL statements to design a functional hostel reservation database and demonstrate its functionality. The SQL implementation includes the creation of all required tables, application of constraints and demonstration of database manipulation through DML and DQL.

2.1 DDL

The implementation of this part is organized inside *Afiq_project1.sql* file, which performs the following tasks:

1. **Database Creation**

   The SQL script begins by creating the main project database and selecting it for use:

```
CREATE DATABASE project_group4;
USE project_group4;
```

Figure 1.1 SQL Script for Database Creation

   This script ensures all the tables and operations are contained and executed within the same database environment.

2. **Table Creation**

   Several tables were created to support the hostel reservation system, including *students*, *room_types*, *room_locations*, *rooms*, *reservations* and *payments*. Each table was designed with appropriate primary keys, data types and constraints. The table below describes each created table:

Table	Primary Key	Foreign Key	Key Attributes	Examples
students	student_id	-	fname	Afiq
			lname	Danish
			ic_number	040718017633

			phone	0172899901
			email	afiqdanish@graduate.utm.my
			faculty	ENG
			gender	Male
room_types	type_id	-	type	Single, Double
			price	450.00
			capacity	2
room_locations	location_id	-	college	Alpha College
			gender_type	Female
rooms	room_id	type_id, location_id	room_number	101
			status	Available
reservations	reservation_id	student_id, room_id	start_date	2025-12-01
			end_date	2026-04-01
			reserve_date	2025-11-26
payments	payment_id	reservation_id	amount	500.00
			method	FPX
			payment_date	2026-01-01

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    fname VARCHAR(50) NOT NULL,
    lname VARCHAR(50) NOT NULL,
    ic_number VARCHAR(20) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(50) NOT NULL,
    faculty VARCHAR(10) NOT NULL,
    gender ENUM('Male', 'Female', 'Not Specified') DEFAULT 'Not Specified'
);
```

Figure 1.2 Snippet of Student Table Creation

3. Constraints Used

i) PRIMARY KEY

- To ensure each table has a unique identifier
- Example: students.student_id, rooms.room_id, reservations.reservation_id

```sql
student_id INT PRIMARY KEY,
```

Figure 1.3 Snippet of Primary Key Implementation

ii) FOREIGN KEY

- To maintain referential integrity between related tables
- Example: students.student_id and rooms.room_id in reservations table
- Foreign keys also include ON DELETE CASCADE and ON DELETE RESTRICT to control their behaviour during deletion

```sql
CONSTRAINT fk_student_id
    FOREIGN KEY(student_id) REFERENCES students(student_id) ON DELETE CASCADE,
CONSTRAINT fk_room_id
    FOREIGN KEY(room_id) REFERENCES rooms(room_id) ON DELETE RESTRICT,
```

Figure 1.4 Snippet of Foreign Key Implementation

iii) UNIQUE

- To ensure no duplicates value for that attribute

- Example: students.ic_number, room_types.type

```
ic_number VARCHAR(20) NOT NULL UNIQUE,
```

Figure 1.5 Snippet of Unique Implementation

iv) NOT NULL

- To ensure all the required fields are not empty

v) CHECK

- To enforce logical rules such as positive room pricing or valid date ranges

```
CONSTRAINT date_chk CHECK (end_date > start_date)
```

Figure 1.6 Snippet of Check Implementation

vi) DEFAULT

- To automate field population when not being inserted with any values

```
reserve_date DATE DEFAULT (CURDATE()),
```

Figure 1.7 Snippet of Default Implementation

2.2 DML

In this section, we focused on inserting actual data into the database, as well as performing updates and deletions to demonstrate the full use of DML operations. After the structure from the DDL stage was completed, we populated each table with sample records so that the reservation system behaves like a real working backend.

1. **Insert Operations (INSERT)**

   We added at least ten records into every main table.For the students table, we used real-looking names and emails based on the list provided. All email addresses follow the format @graduate.utm.my, and the IC numbers start with 04 as required.

   Here is an example of an INSERT statement we used:

```
#Task b
#Inserting 10 data for each tables
INSERT INTO students (student_id, fname, lname, ic_number, phone, email, faculty, gender) VALUES
(1, 'Chew', 'Chiu Xian', '040101-01-1234', '0123456701', 'cxchew@graduate.utm.my', 'ENG', 'Male'),
(2, 'Joanne', 'Ching', '040102-02-2345', '0123456702', 'joanne-321@graduate.utm.my', 'SCI', 'Female'),
```

Figure 2.1 Snippet of Insertion into Students Table

For other tables, such as room_types, room_locations, rooms, reservations, and payments, we inserted records that ensured all foreign keys matched correctly. This allowed the system to later perform meaningful queries such as checking which rooms are full or retrieving payment details for a reservation.

Example INSERT for rooms:

```
INSERT INTO room_types (type, price, capacity) VALUES
('Single', 150.00, 1),
('Double', 250.00, 2),
('Triple', 350.00, 3),
```

Figure 2.2 Snippet of Insertion into room_types

2. **Update Operations (UPDATE)**

To demonstrate how data can be modified after insertion, we performed several update operations.These updates simulate real situations, such as a student changing their phone number or a room becoming occupied after a reservation is made.

Example UPDATE:

```
-- #Updating Student phone number
UPDATE students
SET phone = '0192345890'
WHERE student_id = 1;
```

Figure 2.3 Snippet of Updating Student Number

Another example, marking a room as occupied:

```
-- #updating room status
UPDATE rooms
SET status = 'Occupied'
WHERE room_id IN (101, 108);
```

Figure 2.4 Snippet of Updating Room Status

3. **Delete Operations (DELETE)**

We also included deletion statements to show how records can be removed safely using conditions. Since our tables are linked with foreign keys, we had to ensure that deletions follow the defined constraints.

Example DELETE:

```
-- Delete reservations
DELETE FROM reservations
WHERE end_date < '2025-12-15';
```

Figure 2.5 Snippet of Deleting Reservation

In cases where foreign key restrictions blocked the deletion, we temporarily disabled the checks to allow the data reset process:

```sql
-- Disable safe updates
SET SQL_SAFE_UPDATES = 0;

-- Delete reservations
DELETE FROM reservations
WHERE end_date < '2025-12-15';

-- Re-enable safe updates
SET SQL_SAFE_UPDATES = 1;
```

Figure 2.6 Snippet of Deleting Reservation if there is a Restrictions

These DML operations allowed us to verify that the database works properly under different scenarios and supports realistic system behaviour.

3.0 Query Demonstrations

1. Filtering (WHERE, AND, OR, NOT, BETWEEN, LIKE, NULL)

a. This query filters the list of students to show only female students in the Science faculty.

```
#finding all the female students in science faculty
SELECT *
FROM students
WHERE gender = 'Female' AND faculty = 'SCI';
```

Figure 2.7 Snippet of Filtering Data Using WHERE Conditions

student_id	fname	lname	ic_number	phone	email	faculty	gender
2	Joanne	Ching	040102-02-2345	0123456702	joanne-321@graduate.utm.my	SCI	Female
6	Sabrina	Heng Wei Qi	040106-06-6789	0123456706	sabrinaheng@graduate.utm.my	SCI	Female
10	Nurul	Asyikin Binti Khairul Anuar	040110-10-0123	0123456710	nasyikinkhai@graduate.utm.my	SCI	Female
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.8 Snippet of Output for Filtering Data Using WHERE Conditions

b. This query finds reservations that start between 1 December and 31 December but have an end date earlier than 20 December.

```
#finding reservations between start date 1/12 - 31/12 but end date <20/12
SELECT *
FROM reservations
WHERE start_date BETWEEN '2025-12-01' AND '2025-12-31'
  AND NOT end_date > '2025-12-20';
```

Figure 2.9 Snippet of Filtering Reservations Using BETWEEN and NOT Conditions

reservation_id	student_id	room_id	start_date	end_date	reserve_date
3	3	3	2025-12-03	2025-12-15	2025-11-24
4	4	4	2025-12-05	2025-12-20	2025-11-24
5	5	5	2025-12-06	2025-12-18	2025-11-24
6	6	6	2025-12-07	2025-12-17	2025-11-24
7	7	7	2025-12-08	2025-12-19	2025-11-24
NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.10 Snippet of Output for Filtering Reservations Using BETWEEN and NOT Conditions

2. Sorting (ORDER BY, LIMIT)

a. This query lists all students sorted by last name in ascending order and first name in descending order.

```
#a)listing all the students ordered by last name ascending and first name descending
SELECT *
FROM students
ORDER BY lname ASC, fname DESC;
```

Figure 2.11 Snippet of Sorting Students Using ORDER BY Clause

student_id	fname	lname	ic_number	phone	email	faculty	gender
4	Lubna	Al Haani binti Radzuan	040104-04-4567	0123456704	haani1224@graduate.utm.my	ART	Female
9	Mohamed	Alif Fathi bin Abdul Latif	040109-09-9012	0123456709	aliffathi@graduate.utm.my	ENG	Male
10	Nurul	Asyikin Binti Khairul Anuar	040110-10-0123	0123456710	nasyikinkhai@graduate.utm.my	SCI	Female
3	Cheryl	Cheong	040103-03-3456	0123456703	cheryl322@graduate.utm.my	BUS	Female
2	Joanne	Ching	040102-02-2345	0123456702	joanne-321@graduate.utm.my	SCI	Female
1	Chew	Chiu Xian	040101-01-1234	0192345890	cxchew@graduate.utm.my	ENG	Male
8	Evelyn	Goh	040108-08-8901	0123456708	evelynngoh@graduate.utm.my	ART	Female
6	Sabrina	Heng Wei Qi	040106-06-6789	0123456706	sabrinaheng@graduate.utm.my	SCI	Female
7	Teh	Ru Qian	040107-07-7890	0123456707	tehruqian@graduate.utm.my	BUS	Female
5	Chau	Ying Jia	040105-05-5678	0123456705	chauyingjia@graduate.utm.my	ENG	Female
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.12 Snippet of Output for Sorting Students Using ORDER BY Clause

b. This query displays the top five most expensive room types by ordering them from highest to lowest price.

```
#b)Show top 5 most expensive room types
SELECT *
FROM room_types
ORDER BY price DESC
LIMIT 5;
```

Figure 2.13 Snippet of Sorting Room Types by Price Using ORDER BY and LIMIT

type_id	type	price	capacity
10	Family	1200.00	4
6	Suite	1000.00	3
9	Premium	800.00	2
5	Deluxe	600.00	2
4	Quad	450.00	4
NULL	NULL	NULL	NULL

Figure 2.14 Snippet of Output for Sorting Room Types by Price Using ORDER BY and LIMIT

3. Aggregation (COUNT, SUM, AVG, MAX, MIN)

a. This query counts how many students belong to each faculty.

```
#a)count total students in faculty
SELECT faculty, COUNT(*) AS total_students
FROM students
GROUP BY faculty;
```

Figure 2.15 Snippet of Counting Total Students in Each Faculty

faculty	total_students
ENG	3
SCI	3
BUS	2
ART	2

Figure 2.16 Snippet of Output for Counting Total Students in Each Faculty

b. This query calculates the total revenue collected from all payments.

```
#b)Total payment amount collected for all reservations
SELECT SUM(amount) AS total_revenue
FROM payments;
```

Figure 2.17 Snippet of Calculating Total Payment Amount Using SUM Function

total_revenue
4700.00

Figure 2.18 Snippet of Output for Calculating Total Payment Amount Using SUM Function

c. This query computes the average price for each room type.

```
#c)Average room price by type
SELECT type, AVG(price) AS avg_price
FROM room_types
GROUP BY type;
```

Figure 2.19 Snippet of Calculating Average Room Price by Type Using AVG Function

type	avg_price
Deluxe	600.000000
Double	250.000000
Economy	100.000000
Family	1200.000000
Premium	800.000000
Quad	450.000000
Single	150.000000
Standard	200.000000
Suite	1000.000000
Triple	350.000000

Figure 2.20 Snippet of Output for Calculating Average Room Price by Type Using AVG Function

4. Grouping and filtering groups (GROUP BY, HAVING)

a. This query counts how many rooms exist in each location (college). Every location_id has exactly one room, so the total_rooms value is 1 for all entries.

```
#a.Group by - count how many rooms exists in each location (college)
SELECT location_id, COUNT(*) AS total_rooms
FROM rooms
GROUP BY location_id;
```

Figure 2.21 Snippet of Counting Rooms in Each Location Using GROUP BY

location_id	total_rooms
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1

Figure 2.22 Snippet of Output for Counting Rooms in Each Location Using GROUP BY

b. This query shows only locations that have more than one room. Since every location in the rooms table has exactly one room, there is no location with total_rooms > 1, so the output is blank.

```
#b.Group by + Having - Show only locations (colleges) that have more than 1 room
SELECT location_id, COUNT(*) AS total_rooms
FROM rooms
GROUP BY location_id
HAVING COUNT(*) > 1;
```

Figure 2.23 Snippet of Filtering Locations Using GROUP BY and HAVING

	location_id	total_rooms

Figure 2.24 Snippet of Output for Filtering Locations Using GROUP BY and HAVING

5. Numeric and string functions (ROUND, TRUNCATE, UPPER, LENGTH, CONCAT,SUBSTR)

a. This query demonstrates multiple string and numeric functions, including joining names, converting text to uppercase, counting characters, extracting substrings, and formatting room prices using ROUND and TRUNCATE.

```
#Combination of (ROUND, TRUNCATE, UPPER, LENGTH, CONCAT, SUBSTR)
SELECT
    -- joins first name and last name
    CONCAT(fname, ' ', lname) AS full_name,
    -- converts faculty to uppercase
    UPPER(faculty) AS faculty_uppercase,
    -- counts characters in email
    LENGTH(email) AS email_length,
    -- extracts first 3 letters of first name
    SUBSTR(fname, 1, 3) AS fname_3letters,
    -- round room price(nearest whole number)
    ROUND(rt.price) AS rounded_price,
    -- truncate room price (no decimals)
    TRUNCATE(rt.price, 0) AS truncated_price
FROM students s
JOIN reservations r ON s.student_id = r.student_id
JOIN rooms rm ON r.room_id = rm.room_id
JOIN room_types rt ON rm.type_id = rt.type_id;
```

Figure 2.25 Snippet of Using Numeric and String Functions in a Combined Query

full_name	faculty_uppercase	email_length	fname_3letters	rounded_price	truncated_price
Cheryl Cheong	BUS	25	Che	350	350
Lubna Al Haani binti Radzuan	ART	25	Lub	450	450
Chau Ying Jia	ENG	27	Cha	600	600
Sabrina Heng Wei Qi	SCI	27	Sab	1000	1000
Teh Ru Qian	BUS	25	Teh	200	200
Evelyn Goh	ART	26	Eve	100	100
Mohamed Alif Fathi bin Abdul Latif	ENG	25	Moh	800	800
Nurul Asyikin Binti Khairul Anuar	SCI	28	Nur	1200	1200

Figure 2.26 Snippet of Output for Numeric and String Functions in Combined Query

6. Conditional logic (CASE WHEN)

This query assigns each student into a category which is Engineering Student, Science Student, or Other Faculty based on the value in the faculty column.

```
#Categorize students based on faculty
SELECT student_id, fname, lname, faculty,
    CASE
        WHEN faculty = 'ENG' THEN 'Engineering Student'
        WHEN faculty = 'SCI' THEN 'Science Student'
        ELSE 'Other Faculty'
    END AS student_category
FROM students;
```

Figure 2.27 Snippet of Categorizing Students Based on Faculty Using CASE WHEN

student_id	fname	lname	faculty	student_category
1	Chew	Chiu Xian	ENG	Engineering Student
2	Joanne	Ching	SCI	Science Student
3	Cheryl	Cheong	BUS	Other Faculty
4	Lubna	Al Haani binti Radzuan	ART	Other Faculty
5	Chau	Ying Jia	ENG	Engineering Student
6	Sabrina	Heng Wei Qi	SCI	Science Student
7	Teh	Ru Qian	BUS	Other Faculty
8	Evelyn	Goh	ART	Other Faculty
9	Mohamed	Alif Fathi bin Abdul Latif	ENG	Engineering Student
10	Nurul	Asyikin Binti Khairul Anuar	SCI	Science Student

Figure 2.28 Snippet of Output for Categorizing Students Based on Faculty Using CASE WHEN

7. Subqueries (single-row, multiple-row, correlated)

a. This query retrieves payments that are greater than the average payment amount across all payment records.

```
#a.Single-row (returns 1 value) - compare with average amount
SELECT *
FROM payments
WHERE amount >
    (SELECT AVG(amount) FROM payments);
```

\

Figure 2.29 Snippet of Single-Row Subquery Comparing Payment Amount with Average Amount

payment_id	reservation_id	amount	method	payment_date
5	5	600.00	CARD	2025-11-24
6	6	1000.00	OTHER	2025-11-24
9	9	800.00	CARD	2025-11-24
10	10	1200.00	OTHER	2025-11-24
NULL	NULL	NULL	NULL	NULL

Figure 2.30 Snippet of Output for Single-Row Subquery Comparing Payment Amount with Average Amount

b. This query returns all students whose student_id appears in the reservations table, meaning they have at least one reservation.

```
#b.Multiple-row(returns many rows) - find students who have made reservations
SELECT *
FROM students
WHERE student_id
IN
    (SELECT student_id FROM reservations);
```

Figure 2.31 Snippet of Multiple-Row Subquery to Find Students with Reservations

student_id	fname	lname	ic_number	phone	email	faculty	gender
3	Cheryl	Cheong	040103-03-3456	0123456703	cheryl322@graduate.utm.my	BUS	Female
4	Lubna	Al Haani binti Radzuan	040104-04-4567	0123456704	haani1224@graduate.utm.my	ART	Female
5	Chau	Ying Jia	040105-05-5678	0123456705	chauyingjia@graduate.utm.my	ENG	Female
6	Sabrina	Heng Wei Qi	040106-06-6789	0123456706	sabrinaheng@graduate.utm.my	SCI	Female
7	Teh	Ru Qian	040107-07-7890	0123456707	tehruqian@graduate.utm.my	BUS	Female
8	Evelyn	Goh	040108-08-8901	0123456708	evelynngoh@graduate.utm.my	ART	Female
9	Mohamed	Alif Fathi bin Abdul Latif	040109-09-9012	0123456709	aliffathi@graduate.utm.my	ENG	Male
10	Nurul	Asyikin Binti Khairul Anuar	040110-10-0123	0123456710	nasyikinkhai@graduate.utm.my	SCI	Female
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.32 Snippet of Output for Multiple-Row Subquery to Find Students with Reservations

c. This query checks each student row and returns only those who have a matching reservation, using EXISTS to test the relationship row-by-row.

```
#c.Correlated - show students who have at least one reservation
SELECT s.*
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM reservations r
    WHERE r.student_id = s.student_id
);
```

Figure 2.33 Snippet of Correlated Subquery to Show Students with at Least One Reservation

student_id	fname	lname	ic_number	phone	email	faculty	gender
3	Cheryl	Cheong	040103-03-3456	0123456703	cheryl322@graduate.utm.my	BUS	Female
4	Lubna	Al Haani binti Radzuan	040104-04-4567	0123456704	haani1224@graduate.utm.my	ART	Female
5	Chau	Ying Jia	040105-05-5678	0123456705	chauyingjia@graduate.utm.my	ENG	Female
6	Sabrina	Heng Wei Qi	040106-06-6789	0123456706	sabrinaheng@graduate.utm.my	SCI	Female
7	Teh	Ru Qian	040107-07-7890	0123456707	tehruqian@graduate.utm.my	BUS	Female
8	Evelyn	Goh	040108-08-8901	0123456708	evelynngoh@graduate.utm.my	ART	Female
9	Mohamed	Alif Fathi bin Abdul Latif	040109-09-9012	0123456709	aliffathi@graduate.utm.my	ENG	Male
10	Nurul	Asyikin Binti Khairul Anuar	040110-10-0123	0123456710	nasyikinkhai@graduate.utm.my	SCI	Female
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.34 Snippet of Output for Correlated Subquery to Show Students with at Least One Reservation

8. Set operations (UNION, NOT EXISTs)

a. This query merges two result sets which is male students and female students into one combined list using UNION.

```
#a. Union - combine two set which is male and female student
SELECT fname, lname
FROM students
WHERE gender = 'Male'
UNION
SELECT fname, lname
FROM students
WHERE gender = 'Female';
```

Figure 2.35 Snippet of Combining Male and Female Students Using UNION

fname	lname
Chew	Chiu Xian
Mohamed	Alif Fathi bin Abdul Latif
Joanne	Ching
Cheryl	Cheong
Lubna	Al Haani binti Radzuan
Chau	Ying Jia
Sabrina	Heng Wei Qi
Teh	Ru Qian
Evelyn	Goh
Nurul	Asyikin Binti Khairul Anuar

Figure 2.36 Snippet of Output for Combining Male and Female Students Using UNION

b. This query returns students who do not appear in the reservations table, meaning they have never made a reservation.

```
#b. Not Exists - show student who dont have any reservation
SELECT *
FROM students s
WHERE NOT EXISTS (
    SELECT 1
    FROM reservations r
    WHERE r.student_id = s.student_id
);
```

Figure 2.37 Snippet of Showing Students Without Any Reservations Using NOT EXISTS

student_id	fname	lname	ic_number	phone	email	faculty	gender
1	Chew	Chiu Xian	040101-01-1234	0192345890	cxchew@graduate.utm.my	ENG	Male
2	Joanne	Ching	040102-02-2345	0123456702	joanne-321@graduate.utm.my	SCI	Female
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.38 Snippet of Output for Showing Students Without Any Reservations Using NOT EXISTS

9. Joins (NATURAL, INNER, LEFT OUTER, SELF)

a. This query links reservations and payments automatically using their shared column, reservation_id.

```
#a. NATURAL JOIN - reservations and paymenrs share reservation_id
SELECT *
FROM reservations
NATURAL JOIN payments;
```

Figure 2.39 Snippet of NATURAL JOIN Between Reservations and Payments

reservation_id	student_id	room_id	start_date	end_date	reserve_date	payment_id	amount	method	payment_date
3	3	3	2025-12-03	2025-12-15	2025-11-24	3	350.00	CASH	2025-11-24
4	4	4	2025-12-05	2025-12-20	2025-11-24	4	450.00	FPX	2025-11-24
5	5	5	2025-12-06	2025-12-18	2025-11-24	5	600.00	CARD	2025-11-24
6	6	6	2025-12-07	2025-12-17	2025-11-24	6	1000.00	OTHER	2025-11-24
7	7	7	2025-12-08	2025-12-19	2025-11-24	7	200.00	CASH	2025-11-24
8	8	8	2025-12-09	2025-12-22	2025-11-24	8	100.00	FPX	2025-11-24
9	9	9	2025-12-10	2025-12-21	2025-11-24	9	800.00	CARD	2025-11-24
10	10	10	2025-12-11	2025-12-23	2025-11-24	10	1200.00	OTHER	2025-11-24

Figure 2.40 Snippet of Output for NATURAL JOIN Between Reservations and Payments

b. This query displays reservation details along with each student's full name by matching student_id in both tables.

```
#b.INNER JOIN - show reservations with student name
SELECT
    r.reservation_id,
    CONCAT(s.fname, ' ',s.lname) AS student_name,
    r.room_id,
    r.start_date
FROM reservations r
INNER JOIN students s ON s.student_id = r.student_id;
```

Figure 2.41 Snippet of INNER JOIN to Show Reservations with Student Names

reservation_id	student_name	room_id	start_date
3	Cheryl Cheong	3	2025-12-03
4	Lubna Al Haani binti Radzuan	4	2025-12-05
5	Chau Ying Jia	5	2025-12-06
6	Sabrina Heng Wei Qi	6	2025-12-07
7	Teh Ru Qian	7	2025-12-08
8	Evelyn Goh	8	2025-12-09
9	Mohamed Alif Fathi bin Abdul Latif	9	2025-12-10
10	Nurul Asyikin Binti Khairul Anuar	10	2025-12-11

Figure 2.42 Snippet of Output for INNER JOIN to Show Reservations with Student Names

c. This query lists every room, including those without reservations. NULL appears for rooms with no matching reservation.

```
#c.LEFT OUTER - show all rooms, even if not reserved
SELECT
    rm.room_id,
    rm.room_number,
    r.reservation_id
FROM rooms rm
LEFT JOIN reservations r ON rm.room_id = r.room_id;
```

Figure 2.43 Snippet of LEFT OUTER JOIN to Show All Rooms Even If Not Reserved

room_id	room_number	reservation_id
1	101	NULL
2	102	NULL
3	103	3
4	104	4
5	105	5
6	106	6
7	107	7
8	108	8
9	109	9
10	110	10

Figure 2.44 Snippet of Output for LEFT OUTER JOIN to Show All Rooms Even If Not Reserved

d. This query compares the rooms table to itself to find pairs of rooms that share the same type_id. The output is empty because every room in the rooms table has a unique type_id. Since no two rooms share the same type, the SELF JOIN condition (matching rooms with the same type_id) produces no results.

```
#d.SELF JOIN - find rooms with the same type
SELECT
    a.room_id AS roomA,
    b.room_id AS roomB,
    a.type_id
FROM rooms a
JOIN rooms b ON a.type_id = b.type_id
WHERE a.room_id <> b.room_id;
```

Figure 2.45 Snippet of SELF JOIN to Find Rooms with the Same Room Type

	roomA	roomB	type_id

Figure 2.46 Snippet of Output for SELF JOIN to Find Rooms with the Same Room Type

4.0 Optimization Section

4.1 Before Indexing

1. **BTREE Performance**

```
# -- PART D – INDEXING & OPTIMIZATION
-- 1. BEFORE INDEXING
-- BTREE TEST (price > 200)
EXPLAIN ANALYZE
SELECT rm.room_number, rt.type, rt.price
FROM rooms rm
JOIN room_types rt ON rm.type_id = rt.type_id
WHERE rt.price > 200.00;
```

Figure 2.47 Snippet of EXPLAIN ANALYZE for Price Filtering before BTREE Indexing

EXPLAIN
-> Nested loop inner join (cost=4.75 rows=3.3...

Figure 2.48 Snippet of ANALYZE Output Showing Query Cost before indexing

Based on the query and output provided, the EXPLAIN ANALYZE statement evaluates a query that joins the rooms and room_types tables to find rooms with a price greater than 200.00. The output shows a "Nested loop inner join" with a cost of 4.75. This indicates that since there is no index on the price column yet, the database has to perform a full table scan on room_types. It inspects every row for the condition instead of directly retrieving the matching records.

2. **FULLTEXT Performance**

```
-- FULLTEXT TEST -> college LIKE '%College%'
EXPLAIN ANALYZE
SELECT rl.college, rm.room_number
FROM room_locations rl
JOIN rooms rm ON rl.location_id = rm.location_id
WHERE rl.college LIKE '%College%';
```

Figure 2.49 Snippet of EXPLAIN ANALYZE for Text Search Using LIKE Operator before indexing

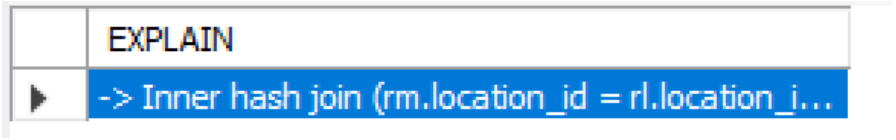

Figure 2.50 Snippet of EXPLAIN ANALYZE Output showing query cost before indexing

According to the query and output provided, the EXPLAIN ANALYZE results show that using the LIKE operator with a leading wildcard ('%College%') to filter text is inefficient. This pattern stops the database from using regular indexes. As a result, it requires a full table scan, meaning every row in the room_locations table is checked one by one to find matches. The "Inner hash join" confirms that the database processes the data without a good text search strategy, pointing out the performance issue that calls for a FULLTEXT index.

4.2 Indexes Creation

1. **BTREE Index on price**

```
-- 2. CREATE INDEXES
-- BTREE INDEX on price column
CREATE INDEX idx_price_btree
ON room_types(price);
```

Figure 2.51 Snippet of SQL Command to Create BTREE Index on price column

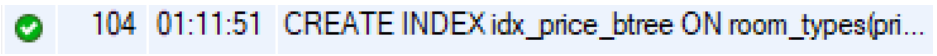

Figure 2.52 Snippet of Output for creating BTREE Index on price column

Based on the query and output provided, the command CREATE INDEX idx_price_btree ON room_types(price); creates a B-Tree index on the price column of the room_types table. The green checkmark in the execution log confirms that this index was created successfully. This step is necessary to fix the full table scan issue mentioned earlier. By indexing this column, the database can now find records based on price values more efficiently, without checking every row in the table.

2. **FULLTEXT Index on College**

```
-- 2.2 FULLTEXT INDEX on college column
CREATE FULLTEXT INDEX idx_college_fulltext
ON room_locations(college);
```

Figure 2.53 Snippet of SQL Command to Create FULLTEXT Index on college column

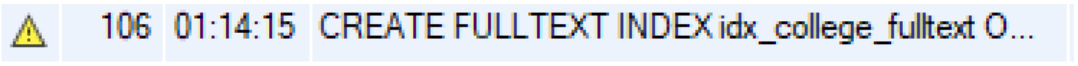

Figure 2.52 Snippet of Output for creating FULLTEXT Index on college column

Based on the query and output provided, the command CREATE FULLTEXT INDEX idx_college_fulltext ON room_locations(college) is run to create an index designed for efficient text searching on the college column. This step directly addresses the slow performance of the LIKE operator mentioned earlier. The FULLTEXT index tokenizes text content, enabling quick keyword lookups instead of scanning every row. The output log verifies the execution of this command, getting the database ready to manage future search queries with much faster speed.

4.3 After Indexing

1. **BTREE Performance**

```
-- 3. AFTER INDEXING
-- BTREE QUERY AFTER INDEXING
EXPLAIN ANALYZE
SELECT rm.room_number, rt.type, rt.price
FROM rooms rm
JOIN room_types rt ON rm.type_id = rt.type_id
WHERE rt.price > 200.00;
```

Figure 2.53 Snippet of EXPLAIN ANALYZE for price filtering after BTREE Indexing

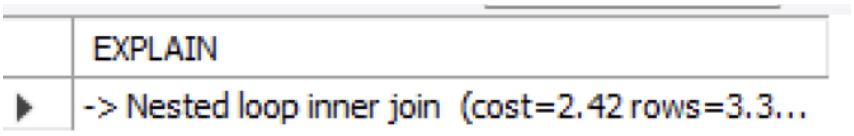

Figure 2.54 Snippet of ANALYZE Output Showing Query Cost indexing

Based on the text and query analysis, creating the idx_price_btree index improved the query performance. It allowed the database to use a "range access type" strategy rather than a full table scan. This change helped the optimizer find records where the price is greater than 200.00. As a result, the estimated query cost dropped significantly from 4.75 to 2.42, and the need for extensive table checks was eliminated.

2. FULLTEXT Performance

```
-- FULLTEXT QUERY AFTER INDEXING
EXPLAIN ANALYZE
SELECT rl.college, rm.room_number
FROM room_locations rl
JOIN rooms rm ON rl.location_id = rm.location_id
WHERE MATCH(rl.college) AGAINST ('College' IN NATURAL LANGUAGE MODE);
```

Figure 2.55 Snippet of EXPLAIN ANALYZE for TEXT Search using MATCH AGAINST after indexing

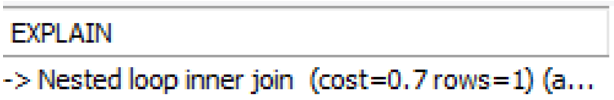

Figure 2.56 Snippet of EXPLAIN ANALYZE Output showing query cost after indexing

Based on the query and output provided, using the MATCH(...) AGAINST(...) syntax allows the database to take advantage of the FULLTEXT index on the college column. This leads to a significant improvement in performance. The EXPLAIN ANALYZE output shows a low query cost of 0.7 and a shift to a "Nested loop inner join." This confirms that the database does not have to scan every row as it did with the LIKE operator. Instead, it quickly finds the specific keyword tokens through the index, proving that the optimization worked.

3. FULLTEXT Search Demo Query

```
-- FULLTEXT Search Demo
SELECT rl.college, rm.room_number
FROM room_locations rl
JOIN rooms rm ON rl.location_id = rm.location_id
WHERE MATCH(rl.college) AGAINST ('College' IN NATURAL LANGUAGE MODE);
```

Figure 2.57 Snippet of FULLTEXT Search Demo Query

college	room_number
Alpha College	101
Beta College	102
Gamma College	103
Delta College	104
Epsilon College	105
Zeta College	106
Theta College	107
Iota College	108
Kappa College	109
Lambda College	110

Figure 2.58 Snippet of Result of FULLTEXT Search Query

Based on the query and output given, this section shows how to do a FULLTEXT search using the MATCH(...) AGAINST(...) syntax in Natural Language Mode. The query filters the room_locations table for the keyword "College," pulling a specific list of rooms and their related college names, such as "Alpha College" and "Beta College." This result confirms that the FULLTEXT index works correctly. It lets the database find records with the specified text without checking unrelated data.

4.4 Show indexes

```
-- SHOW created indexes
SHOW INDEX FROM room_types;
SHOW INDEX FROM room_locations;
```

Figure 2.59 Snippet of SHOW INDEX query

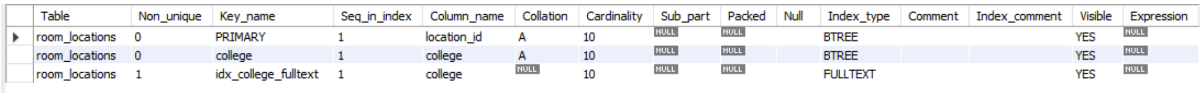

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
room_locations	0	PRIMARY	1	location_id	A	10	NULL	NULL		BTREE			YES	NULL
room_locations	0	college	1	college	A	10	NULL	NULL		BTREE			YES	NULL
room_locations	1	idx_college_fulltext	1	college	NULL	10	NULL	NULL		FULLTEXT			YES	NULL

Figure 2.60 Snippet of SHOW INDEX Output Verifying Created Indexes

Based on the query and output provided, this step acts as a verification process to ensure that the intended improvements have been correctly applied to the database schema. Running SHOW INDEX confirms that the idx_college_fulltext index exists on the room_locations table with the specific FULLTEXT index type. This shows that the database is now set up to handle efficient keyword searches on the college column. The accompanying text also confirms the simultaneous creation of the idx_price_btree index for numeric improvements.

5.0 Team Work

1. **Muhammad Afiq Danish bin Mohd Hazni (Database Designer)**

   As the database designer, my role was to develop the structure of the hostel reservation system using DDL. I created all the necessary tables, defined their attributes and applied constraints such as primary keys, foreign keys, unique fields, default values and check conditions to ensure the accuracy and consistency of the stored data. I also ensured that the relationship between tables were properly connected so that the student, reservation and room data could work together smoothly. Throughout this project, I learned how to design a well organized relational database, implement constraints that reflect real-world rules and structure the backend in a way that supports the whole operations.

2. **Muhammad Syahmi Faris bin Rusli (Query Specialist DQL)**

   As the Query Specialist, my role was to develop and test all DSL queries used in the project. I created the filtering, sorting, aggregation, string and numeric functions, conditional logic, subqueries, set operations, and join queries needed to demonstrate how the database works. I also generated the outputs and explanations for each query to verify that the tables, relationships, and constraints were functioning as expected. Through this task, I strengthened my understanding of retrieving and manipulating data across multiple tables and ensuring that the system returned accurate and meaningful results based on the values that my friend Dhesh has inserted into respective tables.

3. **Dheshieghan A/L Saravana Moorthy (Data Manipulation & Query Developer (DML + DQL))**

   For this project, my job was mainly to handle the data side of the database. I was the one who inserted all the sample records into the tables and made sure everything lined up properly with the foreign keys, because even one small mistake can break the whole thing. After getting the data in place, I worked on the update and delete statements to show how the system would change or remove information in real situations. On top of that, I also did several DQL queries, especially the ones involving filtering, sorting, and basic aggregation. These helped us check whether the data behaved the way it was supposed to and whether the tables were linked correctly. Doing this part taught me a lot about how sensitive databases can be, and how

important it is to structure and manage the data carefully so that everything runs smoothly.

4. **Pravinraj A/L Sivabathi (Indexing & Optimization Analyst)**

   As the Indexing and Optimization Analyst, my main job was to ensure the system's performance and scalability by finding bottlenecks and implementing efficient search strategies. I used the EXPLAIN ANALYZE command to check query execution plans, where I identified problems like Full Table Scans when filtering by price or searching for text. To fix these issues, I created a BTREE index to improve numeric range queries and a FULLTEXT index to replace slow LIKE pattern matching with efficient natural language searches. I documented the "Before vs. After" performance metrics, showing that these changes significantly lowered query costs and improved execution speeds. This role gave me a deep understanding of database internals. I learned how execution plans work and the crucial role of indexing in managing large datasets effectively.

6.0 Conclusion

In conclusion, this project allowed us to apply the full range of SQL concepts learned throughout the course in a practical and meaningful way. By developing a hostel reservation system, we designed a complete backend that supports real-world operations such as storing student information, managing room details, processing reservations, and handling payments. Through the DDL phase, we built a well-structured and normalized database that ensures data accuracy and proper relationships between entities. The DML and DQL components further demonstrated how the system behaves with actual data, enabling us to retrieve, update, and manipulate information using various SQL techniques including filtering, sorting, aggregation, functions, conditional logic, subqueries, and joins.

The optimization section provided us with deeper insight into how database performance can be improved using indexing. By analyzing queries before and after applying BTREE and FULLTEXT indexes, we observed clear reductions in query cost and more efficient execution plans. This experience highlighted the importance of indexing in supporting scalability and faster data retrieval, especially when dealing with larger datasets.

Overall, this project strengthened our understanding of how a functional database system is designed, used, and optimized. Each team member contributed to different parts of the system, and through this collaboration, we gained valuable hands-on experience in creating a reliable backend that reflects real-life database requirements. This project not only enhanced our SQL skills but also taught us the significance of planning, teamwork, and attention to detail in database development.